Low-Bit Neural Network Quantization for High-Energy Physics Jet Classification: A Reproducible Benchmark of PTQ, QAT, and Fixed-Point Simulation

Piush Kole

October 2026

Abstract

Real-time Level-1 trigger systems at the Large Hadron Collider must classify particle jets within microseconds on FPGA hardware under hard resource ceilings, making neural-network quantization a deployment necessity rather than an optional optimization. We present a reproducible benchmark of post-training quantization (PTQ) and quantization-aware training (QAT) at INT8/INT6/INT4 precision for a jet-tagging multi-layer perceptron (MLP), evaluated on the public `hls4ml_lhc_jets_hlf` dataset. The study covers uniform-precision PTQ/QAT, a second architecture to check generality, a calibration-sample-size sensitivity analysis, a near-exhaustive per-unit mixed-precision search over 2,187 layouts, and a PyTorch-to-fixed-point (`hls4ml`) C-simulation validation. Every reported number is tagged by the tier of evidence that produced it — measured in software, simulated via fixed-point C-simulation, estimated by an analytical heuristic, or not available — and every comparison's selection rule was fixed before the test split was examined. We find that INT8 PTQ is practically lossless, INT4 PTQ degrades substantially but QAT recovers most of the loss, and a near-exhaustive mixed-precision search finds a layout that reliably beats uniform INT6 PTQ at a smaller storage footprint — a conclusion a smaller, hand-picked search had missed. No claim is made about real detector data, physical FPGA performance, or a production trigger system: this is a reproducible software benchmark, not a demonstrated hardware result.

1 Introduction and Motivation

At the Large Hadron Collider, the Level-1 trigger must decide, within microseconds, which of roughly 40 million collisions per second are worth recording (Duarte et al., 2018). Machine-learned classifiers increasingly replace or augment hand-written trigger logic, but any such model must fit within an FPGA's fixed timing and resource budget — making quantization (reducing numeric precision from FP32 to INT8/INT6/INT4) a deployment necessity, not a convenience.

The central empirical question this work asks is simple to state and not simple to answer cleanly: *how much precision can be removed before physics performance becomes unacceptable, and does the answer depend on the quantization strategy, the search granularity, the architecture, or the calibration procedure used to prepare it?* Each of these axes turns out to matter, and the honest answer to "does mixed precision help" is more often "it depends, and here is exactly how" than a clean yes or no.

This work is an independent, non-CERN-affiliated research project. It uses a public, particle-level Monte Carlo simulated dataset, not real detector data, and makes no claim of demonstrated

FPGA synthesis, physical hardware measurement, or production trigger deployment anywhere in this document.

2 Related Work

This project sits in an established HEP-ML lineage. The jet-tagging benchmark used throughout (Section 3) is the hls4ml LHC jet dataset of Duarte et al. (2018), from the same group behind `hls4ml` itself (hls4ml collaboration, 2024), the FPGA-deployment toolchain this work targets. Automatic mixed-precision quantization for this exact niche has advanced substantially in the broader field: QKeras/AutoQKeras (Coelho et al., 2021) pioneered gradient-free per-layer mixed precision validated against real synthesis, and HGQ2 (Sun et al., 2024) is a current state-of-the-art approach, using a gradient-based, per-parameter bit-width search via a differentiable resource proxy. More recent architecture-side work has pushed `hls4ml`-deployable models beyond MLPs toward GNNs and linear-interaction networks for jet tagging (Sun et al., 2025; Moreno et al., 2020), and CMS's WOMBAT system (Moreno et al., 2025) demonstrates a teacher/student design for a real Phase-2-trigger use case. Most relevant to the hardware-validation thread of this work, wa-hls4ml (Hawks et al., 2025) is a very recent public dataset of real `hls4ml`-synthesized networks with pretrained surrogate models; this project trains a small surrogate on the same public data as a cross-check against its own analytical heuristic, rather than treating either as ground truth.

This project's contribution is not a new quantization *method* — it uses standard uniform affine fake-quantization (PTQ) and straight-through-estimator training (QAT), the same mechanisms underlying most of the literature above. The contribution is **benchmarking methodology and rigor**: a near-exhaustive, not hand-picked, mixed-precision search at full statistical power; a dedicated calibration-sensitivity study separating model-seed variance from calibration-set variance; an independently-trained hardware-cost cross-check rather than a single heuristic; and an explicit, pre-registered protocol that prevents test-set selection bias, including disclosure of one earlier result in this same project line that violated that protocol and was consequently excluded from every confirmatory claim (Section 4).

3 Dataset and Preprocessing

The primary dataset is the hls4ml LHC jet-tagging benchmark (Duarte et al., 2018) (OpenML `hls4ml_lhc_jets_hlf` v1, Zenodo DOI 10.5281/zenodo.3602260, CC BY 4.0): 16 hand-engineered high-level substructure features (energy-correlation functions, N-subjettiness ratios, jet-mass variants) per jet, with a 5-class label (gluon, quark, top, W, Z) that is near-balanced (19.4–20.2% per class, confirmed by direct count). The underlying simulation is particle-level Monte Carlo (MadGraph5 + Pythia8 + FastJet anti-k_T, $R = 0.8$, jet p_T 0.8–1.6 TeV, 13 TeV); **no detector simulation is involved, and this is a publicly available simulated dataset, not detector data.**

A 100,000-jet stratified subsample is used throughout, split 70/15/15 into train/validation/test, with every split and the model initialization controlled by one of five seeds $\{42, 123, 2026, 7, 99\}$. All preprocessing (standardization) is fitted on the training split only. Full-dataset SHA-256 fingerprints of the validated feature and label arrays are recorded in the project's reproducibility documentation, so any re-download can be checked byte-for-byte against the exact data used here.

4 Experimental Design and Reproducibility

Every result in this work follows the same discipline:

- **Pre-registration.** A selection rule (which configuration "wins") is written to disk *before* the test split is touched for that comparison.
- **Baseline-before-model.** Non-neural-network baselines are evaluated before any neural result is presented as a headline (Section 4 is reinforced by the mixed-precision and hardware sections below, which both include a non-selected or non-neural reference point).
- **Evidence tiering.** Every reported number is tagged MEASURED, SIMULATED, ESTIMATED, or NOT AVAILABLE, so a reader never has to guess how the number was obtained.
- **The two-test rule.** Every deterministic result is verified by running the same code twice, independently, and diffing the output byte-for-byte.
- **Disclosed bias, not hidden bias.** An earlier exploratory mixed-precision result in this project line (ROC-AUC 0.9278, $n = 3$) was selected using test-split information. Once identified, it was explicitly excluded from every confirmatory claim in this paper and in the project's own dashboard, rather than silently corrected or omitted from the record.

Training uses Adam (learning rate 10^{-3}, weight decay 10^{-4}, batch size 1024), at most 30 epochs with early stopping (patience 6) on validation loss, checkpoint selected by validation loss. QAT fine-tunes 7 epochs ($\max(5, \lceil E/4 \rceil)$) at $0.1\times$ the learning rate from the same-seed FP32 model, using a straight-through estimator so gradients flow through the non-differentiable rounding operation. No hyperparameter was tuned on any split for any architecture. Both PTQ and QAT use uniform affine fake-quantization (quantize-then-dequantize, computed in FP32) rather than real bit-packed integer kernels, since PyTorch's native quantized kernels only support INT8, not INT6/INT4; this is a standard, technically valid research methodology (the same mechanism underlies most QAT literature) but does not produce a packed artifact, so model size is computed analytically (quantized weights at their bit-width, biases at 4 bytes) rather than read from a checkpoint file.

5 Uniform PTQ and QAT Results

Table 1 reports the primary architecture ($16 \to 64 \to 32 \to 32 \to 5$, 4,389 parameters), frozen records, $n = 5$ seeds, macro one-vs-rest test ROC-AUC (MEASURED).

Table 1: Primary architecture, uniform-precision PTQ and QAT.

Configuration	ROC-AUC (mean ± SD)	Δ vs. FP32	Storage (bytes)
FP32	0.9343 ± 0.0011	–	17,556
INT8 PTQ	0.9342 ± 0.0011	-0.0001	4,788
INT6 PTQ	0.9329 ± 0.0014	-0.0014	3,724
INT4 PTQ	0.8819 ± 0.0308	-0.0524	2,660
INT4 QAT	0.9246 ± 0.0018	-0.0097	2,660

INT8 PTQ is practically lossless against FP32. INT6 PTQ costs approximately 0.0014 ROC-AUC. INT4 PTQ degrades substantially (and, as Section 7 shows, is sensitive to calibration procedure), while INT4 QAT recovers most — but not all — of that loss. These frozen records are from

a CUDA-era session; Section 6 re-runs the same protocol on CPU and reports the device difference as observational, not causal.

6 Architecture Comparison

To test whether the pattern above is specific to one architecture, a second MLP ($16 \to 128 \to 64 \to 64 \to 32 \to 5$, 16,837 parameters, roughly 3.8× the parameters of the primary architecture) was evaluated under an identical protocol, on CPU, with both architectures re-run on CPU for a fair comparison (Table 2). Two independent full runs produced identical per-seed metrics and byte-identical checkpoints.

Table 2: Architecture generalization, CPU, $n = 5$ seeds, Δ vs. each architecture's own FP32.

Architecture	FP32	INT8 Δ	INT6 Δ	INT4 PTQ Δ (SD)	QAT recovery
A (4,389 params)	0.9343 ± 0.0015	-0.0001	-0.0010	-0.0438 (0.0122)	78.5%
B (16,837 params)	0.9376 ± 0.0009	0.0000	-0.0010	-0.0264 (0.0045)	73.6%

The qualitative pattern holds in both architectures: INT8 is lossless, INT6 is close to lossless, INT4 PTQ degrades substantially, and INT4 QAT recovers most of that loss. Architecture B's INT4 PTQ loss is numerically smaller (a paired-difference 95% confidence interval of $[-0.0009, +0.0358]$, with 4 of 5 seeds agreeing) — directionally suggestive of a larger model being more robust at 4 bits, but **not statistically established at $n = 5$ seeds**, and reported as such rather than overstated. INT8 and INT6 degradation are statistically indistinguishable between the two architectures. Separately, the primary architecture's CPU re-run (INT4 PTQ 0.8904 ± 0.0122) differs from its original CUDA-era frozen record (INT4 PTQ 0.8819 ± 0.0308, Table 1); this CPU/CUDA difference is **observational only** — a controlled, isolated CUDA-vs-CPU reproduction of the identical protocol was not performed in this environment, so no causal device mechanism is claimed.

7 Calibration Sensitivity

INT4 PTQ's large seed-to-seed spread (Table 1) motivated a dedicated study isolating calibration-subset selection, calibration-set size, model seed, and numerical determinism as candidate explanations. All four candidate mechanisms were tested directly rather than assumed (Table 3).

Table 3: INT4 PTQ accuracy vs. calibration-set size (mean over 5 seeds × 5 draws).

Calibration size (jets)	ROC-AUC	Activation range (A0 hi)	Quantization step	Clip rate
100	0.9073 ± 0.0214	4.88	0.325	0.0005
500	0.8997 ± 0.0257	6.33	0.422	0.0001
1,000	0.8946 ± 0.0283	6.91	0.461	0.0000
5,000	0.8881 ± 0.0290	7.82	0.521	0.0000
10,000	0.8862 ± 0.0279	8.05	0.537	0.0000

Four findings, each directly tested rather than assumed:

1. **Determinism.** Five independent rebuilds of the INT4 variant, and three repeated inferences on the same build, produced bit-identical output (matching SHA-256 hashes). On this CPU, INT4 PTQ is deterministic for a fixed model and calibration set.

2. **Calibration size has a real, directionally consistent, but secondary effect.** Moving from 100 to 10,000 calibration jets costs 0.021 mean ROC-AUC (95% CI $[0.010, 0.033]$, all 5 seeds in the same direction), acting through a wider activation range and a larger quantization step — *not* through clipping, since the clip rate stays at essentially zero at every size (min/max calibration covers the test activations by construction; this mechanism would only activate under percentile or MSE-based clipping, which was not evaluated here).

3. **Model seed dominates the spread.** The between-seed standard deviation of per-seed mean ROC-AUC (0.029) is roughly four times the within-seed, across-calibration-draw standard deviation (0.008). Most of the 0.835–0.909 range observed across seeds is a property of the trained model, not the calibration procedure.

4. **Weight quantization is calibration-free by construction** — weight ranges come from each tensor's own min/max, so only the activation side is affected by calibration-set choice.

8 Mixed-Precision Analysis

An earlier, smaller search over only 9 hand-picked per-unit bit-width layouts concluded there was "no advantage over uniform INT6 PTQ." That search could not have found the result below, because its candidate set did not include it.

A **true near-exhaustive search** — all $3^7 = 2{,}187$ per-unit bit-width combinations ({INT4, INT6, INT8} for each of 7 quantizable units: 4 weight tensors, 3 activation tensors), evaluated at full 5-seed statistical power — finds a genuine Pareto frontier of 23 non-dominated layouts, not a single point (Table 4). The frontier's QAT-fine-tuned selection (storage 2,916 bytes, chosen by a rule pre-registered before the search: highest mean validation ROC-AUC among layouts at or under uniform INT6's storage budget) **reliably beats uniform INT6 PTQ** by +0.0013 mean ROC-AUC (95% CI $[0.0011, 0.0015]$, entirely positive across all 5 seeds), while using 22% less storage, and is statistically indistinguishable from the FP32 baseline.

Table 4: Mixed-precision search result vs. uniform baselines.

Configuration	Storage (bytes)	Mean test ROC-AUC
FP32	17,556	0.9343
INT8 PTQ	4,788	0.9342
INT6 PTQ	3,724	0.9329
INT4 PTQ	2,660	0.8813
INT4 QAT	2,660	0.9246
Mixed PTQ (this search)	2,996	0.9321
Mixed QAT (this search)	**2,916**	**0.9342**

This revises, rather than confirms, the earlier 9-candidate search's conclusion: granularity of the search space materially changes the answer to "does mixed precision help." A genuine engineering reproducibility gap was found and fixed during this work: the per-unit QAT fine-tuning builder did not originally pin a random seed, causing a small ($\sim 3 \times 10^{-4}$ ROC-AUC) but real run-to-run drift inconsistent with the project's reproducibility discipline; this was fixed by seeding explicitly before fine-tuning and re-verified to byte-identical reruns.

9 `hls4ml` Fixed-Point C-Simulation

To check whether the PyTorch-measured quantization results hold under hardware-realistic fixed-point arithmetic, the primary architecture (seed 42, FP32 reference: accuracy 0.7495, ROC-AUC 0.9334) was converted via `hls4ml` 1.3.0 to `ap_fixed<W,6>` (6 integer bits) for $W \in \{8, 10, 12, 14, 16\}$, under four arithmetic configurations, and evaluated by C-simulation (SIMULATED — fixed-point arithmetic, no synthesis, no hardware) against the same 15,000 held-out test jets (Table 5).

Table 5: `hls4ml` C-simulation accuracy by total width W and arithmetic mode (SIMULATED).

W	Default (TRN, WRAP)	Round + saturate	Wide accum.	Round+sat.+wide accum.
8	0.2021	0.5034	0.2017	0.6508
10	0.3541	0.7343	0.5486	0.7471
12	0.7255	0.7489	0.7462	0.7500
14	0.7481	0.7505	0.7507	0.7502
16	0.7511	0.7497	0.7500	0.7498

Two findings, both directly measured: (1) at narrow width (W=8) the default (truncate, wrap) configuration collapses to near-random accuracy, while at $W \geq 14$ every configuration matches the FP32 PyTorch reference (agreement ≥ 0.97) — this is a width effect on a faithfully-converted network, not a conversion failure, since the same network is accurate at wide enough W; (2) rounding and saturation substantially mitigate the narrow-width collapse (e.g. W=8 accuracy rises from 0.20 to 0.65 with rounding, saturation, and a wide accumulator combined), but **the individual contribution of rounding versus saturation versus accumulator width was not separately isolated** — only their combinations were measured, and this limitation is stated explicitly rather than implied away. No HLS synthesis was run and no physical FPGA was measured anywhere in this study (NOT AVAILABLE): no Vivado, Vitis HLS, or Quartus toolchain was available in the experimental environment, so no resource, latency, or initiation-interval number from real synthesis exists. A software heuristic (ESTIMATED) and an ML surrogate trained on the public wa-hls4ml synthesis dataset (Hawks et al., 2025) were used as hardware-resource cross-checks instead of a single unvalidated estimate; the surrogate's own disclosed limitation (near-zero sensitivity to bit-width in its resource predictions) means the analytical heuristic remains the more useful of the two specifically for precision comparisons, even though the surrogate's latency/initiation-interval predictions generalize well (R^2 up to 0.996 on held-out synthesis data).

10 Discussion and Limitations

What this work supports: that a near-exhaustive mixed-precision search materially changes conclusions a smaller, hand-picked search reaches (Section 8); that INT8 PTQ is practically lossless and INT4 PTQ is not, consistently across two architectures (Sections 5–6); that calibration-set size has a real but secondary effect on INT4 PTQ, dominated by model-seed variance (Section 7); and that the PyTorch-measured quantization behavior is corroborated by independent fixed-point C-simulation at sufficient bit width (Section 9).

What this work does not support, explicitly:

- **No physical FPGA performance measurement exists.** Every hardware number in this paper is SIMULATED (C-simulation) or ESTIMATED/surrogate (software heuristics), never MEASURED on real hardware.

- **No completed HLS synthesis result exists.** No Vivado, Vitis HLS, or Quartus toolchain was available; no resource, latency, or initiation-interval number from synthesis is reported anywhere.
- **No claim of demonstrated FPGA speedup.**
- **No claim of real detector data.** The benchmark uses a public, particle-level Monte Carlo-generated jet dataset, with no detector simulation involved (Section 3).
- **No causal conclusion about CPU-versus-CUDA dependence.** The device difference reported in Section 6 is observational; no controlled, isolated experiment establishing a causal device mechanism was run.
- **No claim that rounding and saturation were independently isolated** in the `hls4ml` C-simulation study — only their combinations were measured (Section 9).
- **No claim that this is a production trigger system.** This is a reproducible software benchmark of a classifier, not a deployed, validated, or certified trigger component.
- **No claim of novelty in the quantization method itself**, of CERN affiliation, of publication, or of external peer validation. The mechanisms used (uniform affine PTQ, straight-through-estimator QAT) are standard.
- The exploratory, test-set-selected result (ROC-AUC 0.9278) is excluded from every confirmatory claim in this paper, consistent with the pre-registration discipline of Section 4.
- The Architecture-A-CUDA, Architecture-A-CPU, and Architecture-B-CPU results are kept separate throughout (Tables 1 and 2) and are never pooled or averaged across devices or architectures.

11 Conclusion

This work benchmarks post-training and quantization-aware-training quantization for a jet-tagging classifier with the rigor the eventual deployment stakes warrant: a dedicated calibration-sensitivity study instead of an unexplained seed-to-seed spread, a true near-exhaustive mixed-precision search instead of a hand-picked one, a second architecture to check generality, and an independent fixed-point C-simulation cross-check. INT8 PTQ is practically lossless; INT4 PTQ is not, and its instability is attributable mostly to model-seed variance rather than calibration procedure; a mixed-precision layout found by exhaustive search reliably beats uniform INT6 PTQ at a smaller storage footprint, reversing an earlier, under-searched conclusion. None of this constitutes a demonstrated FPGA result: the practical claim this paper supports is narrower and, we think, more useful — **the search granularity and the calibration procedure both materially change the answer to "how much can this model be quantized," and both should be checked explicitly rather than assumed, before any claim about a specific deployment target is made.**

Reproducibility Statement

Every numerical claim in this paper traces to a specific report and reproduction command in the accompanying public repository (`hep-trigger-quantization-lab`):

- Section 5 → reports/final_results.md
- Section 6 → reports/phase6_architecture_generalization.md
- Section 7 → reports/int4_ptq_stability_analysis.md
- Section 8 → reports/phase12_exhaustive_mixed_precision_pareto.md
- Section 9 → reports/phase7_hls_numerical_validation.md

A full traceability audit of every number in this paper is maintained in paper/CLAIMS_AUDIT.md in the same repository. Pinned dependencies, exact reproduction commands, dataset fingerprints, and the full two-test-rule verification record for every result are documented in docs/REPRODUCIBILITY.md. The repository's test suite (141 tests at time of writing, two independent runs, identical results) covers the data pipeline, quantization code, hardware-estimation path, and the accompanying research dashboard's API layer.

References

Claudionor N. Coelho et al. Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors. *Nature Machine Intelligence*, 3:675–686, 2021.

Javier Duarte et al. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation*, 13:P07027, 2018.

Ben Hawks et al. wa-hls4ml: A public dataset of real hls4ml synthesis results for surrogate resource modeling, 2025.

hls4ml collaboration. hls4ml: FPGA deployment toolchain for machine learning in high-energy physics. https://fastmachinelearning.org/hls4ml/, 2024.

Eric A. Moreno et al. JEDI-net: a jet identification algorithm based on interaction networks. *European Physical Journal C*, 80:58, 2020.

Eric A. Moreno et al. WOMBAT: A trigger-level jet-substructure tagging system, 2025.

Chang Sun et al. HGQ2: Gradient-based mixed-precision quantization for hardware-efficient neural networks, 2024.

Chang Sun et al. JEDI-linear: Jet identification via linear interaction networks, 2025.